

PROFESSIONAL SUMMARY

AI infrastructure engineer with nearly five years of cloud-native AI/HPC experience, focused on production LLM serving since 2024. Owns a distributed inference runtime for heterogeneous accelerator platforms, a Rust KV-aware Router, a P/D Planner, and BatchFlow; contributes to a Kubernetes-native MaaS control plane, the KV Event/Mooncake transfer path, and a LiteLLM gateway. Strong in Go and Python, with Rust engineering experience across model lifecycle, request routing, capacity management, and recovery.

EXPERIENCE

Zhejiang Lab

Aug 2021 - Present

Senior Research Specialist · High-Performance Computing Facility Research Center

2024 - Present

LLM Serving / Model-as-a-Service (MaaS) Platform

- Contributed to the architecture and model-service CRDs of a Kubernetes-native MaaS control plane; implemented controller paths for P/D-disaggregated deployment, prefill/decode worker pools, worker status, and endpoint aggregation, and supported production releases and diagnosis.
- Owned an in-house distributed inference runtime inspired by Dynamo's service-discovery and P/D architecture, implementing SGLang/vLLM engine adapters, worker registration and discovery, and drain-before-scale-in semantics while unifying model publication, P/D bootstrap, and layered readiness.
- Owned a P/D Planner and Rust KV-aware Router: combined multi-cluster load, queue, KV state, and offline capacity profiles for replica planning, then selected P/D workers from prefix KV overlap, real-time load, and health.
- Independently designed and implemented BatchFlow for offline batch inference; contributed to the KV Event/Mooncake path and LiteLLM gateway setup, including unified APIs, authentication, rate limits, monitoring, and cross-component production diagnosis.

Engineering Specialist · Intelligent Supercomputing Research Center

Aug 2021 - 2023

Cloud-Native AI / HPC Platform

- Customized Kubernetes Device Plugin and resource-management paths for heterogeneous accelerator discovery, node resource reporting, container-runtime validation, device-level allocation, and fault isolation.
- Adapted PyTorch workloads, models, and distributed job environments across enterprise Linux distributions and heterogeneous accelerator stacks, establishing the platform foundation for later serving and scheduling work.

SELECTED PRODUCTION SYSTEMS

Model-Serving Control Plane and Distributed Inference Runtime

Declarative Serving · Runtime · Readiness

- Contributed to a unified model-service CRD and controllers that express co-located or P/D-disaggregated topology, worker pools, Services, and Ingresses as one declarative delivery interface; the control plane owns desired state, the runtime publishes actual capability, and the Planner computes replica targets.
- Owned SGLang/vLLM engine adapters, worker registration and discovery, and model publication; used Dynamo as an architectural reference for P/D bootstrap and layered readiness without depending on or modifying Dynamo Runtime.
- Admits newly scaled workers only after complete readiness and registration; on scale-in, drains new traffic before in-flight completion and endpoint unregistration, while isolating failed workers and reducing manual configuration and cross-component diagnosis.

KV-aware Router and KV Event/Mooncake Path

Rust Routing · KV Event · Mooncake Transfer

- Implemented a Rust KV-aware Router that selects prefill/decode workers within a single-model, single-accelerator routing domain from prefix KV overlap, real-time worker load, queue state, and health, with SGLang/vLLM HTTP and gRPC integration.
- Owned the KV Event adapter, ZMQ-to-NATS bridge, Router event consumption, and prefix index, normalizing BlockStored, BlockRemoved, and AllBlocksCleared into Stored, Removed, and Cleared RouterEvents.
- The Router passes bootstrap/transfer context while Mooncake Transfer Engine moves KV tensors directly between P/D workers; the production path supports failed-worker isolation, reselection after transfer anomalies, and stable request completion.

SELECTED PRODUCTION SYSTEMS (CONTINUED)

Multi-Cluster Load-Aware P/D Planner - Aggregates Router, worker, and Prometheus signals across clusters, combining request load, queue depth, KV state, and offline capacity profiles to continuously compute service replica targets. - Plans prefill and decode replicas independently for P/D-disaggregated services and a unified replica target for co-located services; the Planner owns capacity decisions while the control plane and Kubernetes create and place resources. - In production, scales successfully under pressure to reduce queue backlog and restore stable requests, then reclaims idle replicas as load falls to improve cluster utilization.	Capacity Modeling · Replica Autoscaling · Observability
BatchFlow Multi-Tenant Offline Batch Inference - Independently implemented the OpenAI Batch API, durable task state machine, schedulers/workers, streaming JSONL, sharding, tenant/model concurrency controls, cancellation, and output/error writeback to manage the full data-distillation lifecycle. - Uses Redis for shard competition and dispatch while the database remains authoritative for batch, shard, and claim state; Kubernetes Lease provides scheduler leadership, and expired worker claims are reassigned. - Combines shared result files with custom_id-based skipping under at-least-once backend execution and result-level de-duplication, providing recovery, request-level error isolation, and stable long-running operation.	OpenAI Batch API · Scheduling · Recovery

CORE TECHNOLOGIES

Runtime & Data Plane: distributed inference runtime, SGLang, vLLM, KV Event Plane, ZMQ/NATS, Mooncake Transfer Engine, P/D disaggregation
Control Plane & Scheduling: Go, Kubernetes Operator, CRD/controller, worker pools, capacity modeling, P/D replica autoscaling, Device Plugin, Helm, ArgoCD
Routing & Gateway: Rust, KV-aware routing, HTTP/gRPC P/D routing, prefix index, LiteLLM, OpenAI/Anthropic APIs, authentication and rate limits
Batch & Observability: Python, OpenAI Batch API, Redis, MySQL, Kubernetes Lease, Prometheus, QPS/TTFT/TPOT, request tracing

EDUCATION

Arizona State University - M.S. in Computer Science, GPA 3.97 / 4.00, May 2019 - Dec 2020
Arizona State University - B.S. in Computer Science, Aug 2015 - May 2019

PUBLICATION & INTELLECTUAL PROPERTY

Prompt2Length: A Lightweight Model for Predicting Output Length in LLM Inference - third author, IEEE AIAHPC 2025; co-inventor on two granted invention patents and named contributor on two registered software copyrights.

LANGUAGES

Mandarin Chinese (native); English (full professional proficiency).